STUDENT HANDOUT · DAY 1

Digital Forensics & Cyber Security
Day 1 — Foundations, Linux & Networks

Saturday 24 October 2026 · 09:00–17:00 · Atomcamp programme

Trainer: Waseem Abbas Khan — Senior Cybersecurity Specialist, CEI Master Trainer

What this handout is

Everything we covered today, written down properly and expanded. The classroom moves fast, and notes taken at speed always lose detail. Use this to revise, to redo the labs at home, and to prepare for the quiz next session.

Every command in this handout was tested on Kali before it was written here. Where something is easy to get wrong, it is marked TRAP. Where a fact needs a correction from the live session, it is marked CORRECTION.

What we covered today

#	Topic	What you can do now
1	What information security actually is	Explain CIA, MITM, hashes and DoS with a real example each
2	The Cyber Kill Chain	Recite the 7 stages and name what the defender does at each one
3	Virtualisation and your lab	Explain host, guest, hypervisor, and why isolation matters
4	Linux distributions and the shell	Read the prompt, switch users, escalate properly
5	Files, permissions and users	Read rwx and octal, use chmod and adduser correctly
6	Package management and services	Install, start, stop and inspect a service such as Apache
7	Processes and logs	Find, inspect and kill a process; read Apache and system logs
8	Networking basics	Read an IP, identify an address class, know the common ports, use ping/TTL
9	Ciphers, encoding and encryption	Explain Caesar, Base64, hashing, AES and RSA, and use OpenSSL
10	The security vocabulary	Define asset, threat, vulnerability, risk and exploit correctly
11	Shell essentials	Use ; | && &, help flags, aliases and shell history

HOW TO USE THIS TONIGHT Read this tonight, then redo the commands in Section 7 (‘Do it again at home’) in your own VM. Repeating a command from a handout without the trainer is the only way to find out whether you actually learned it.

1. What information security actually is

1.1 The CIA triad, with the diagram from the board

Confidentiality, Integrity, Availability. These are the three things you are protecting. Every control in the world exists to protect one of them, and almost every incident damages at least one.

Letter	Plain meaning	Breach example	The control that protects it
Confidentiality	Only the right people can see it	Equifax 2017: unpatched Apache Struts on a public portal, roughly 147 million people's records read by attackers	Patching, segmentation, encryption
Integrity	It has not been changed behind your back	Stuxnet 2010: PLC commands manipulated while operators were shown normal readings	Signatures, hashes, out-of-band verification
Availability	It works when you need it	Colonial Pipeline 2021: ransomware forced the operator to shut the pipeline itself	Redundancy, backups, MFA on remote access

THE POINT The three examples share one feature: the control that would have prevented them was already known and cheap. The failure was organisational, not technical. Remember that when you write recommendations later in this course.

1.2 MITM, drawn properly

Man-in-the-middle means the attacker sits between the client and the server and relays the conversation while reading or modifying it. Drawn on the board today as:

```
CLIENT  ---->  ATTACKER (MITM)  ---->  SERVER
      <----              <----
```

What makes MITM possible in practice: an unencrypted protocol, a network position you control (rogue Wi-Fi, ARP spoofing, DNS poisoning), or a TLS interception that the user accepted. What stops it: TLS with proper certificate validation, certificate pinning in mobile apps, HSTS, and not clicking through certificate warnings.

MITM variant	How the attacker wins	How the defender breaks it
Rogue Wi-Fi hotspot	Clients connect to the attacker's access point	TLS verification; user training; never trust open Wi-Fi
ARP spoofing on a LAN	Attacker answers ARP for the gateway and relays traffic	Dynamic ARP inspection; port security
TLS interception with a fake CA	User installs an attacker CA or clicks through a warning	Certificate pinning; HSTS preload; reporting
DNS poisoning	Answers point the victim at the attacker's server	DNSSEC where available; DoH; monitoring

1.3 Hashes — one-directional by design

A hash function takes any input and produces a fixed-length output. Three properties matter:

- **Deterministic:** Same input always gives the same hash.
- **One-way:** You cannot go backwards from hash to input. That is the 'one-sided' note from the board.
- **Collision-resistant:** Two different inputs must not give the same hash. A break in this property is what ended MD5 and SHA-1 for security use.

```
echo -n 'hello' | sha256sum
echo -n 'hello' | md5sum
echo -n 'Hello' | sha256sum    # one letter changes everything
```

OUTPUT

```
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824 -
5d41402abc4b2a76b9719d911017c592 -
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969 -
```

Where you will use this in the course: verifying that an evidence image was not altered (weekend 8), and confirming a file you downloaded is the file you expected.

TRAP A hash is not encryption. You cannot 'decrypt' a hash. When a site is 'cracked', the attacker guessed candidate passwords, hashed each guess, and compared. That is brute force, not decryption.

1.4 DoS and DDoS

Denial of service means making a service unavailable to its real users. It targets Availability. Distributed DoS uses many sources so that blocking one address does nothing.

Type	Example	Signature	Defence
Volumetric	Bandwidth flood	Uplink saturated	Scrubbing, upstream filtering
Protocol	SYN flood	Half-open connections exhausted	SYN cookies, rate limits
Application layer	HTTP request flood against a search page	CPU-expensive endpoint hammered	Caching, rate limits, CAPTCHA, WAF

SCOPE DoS against your own lab VM is allowed and is a legitimate lab exercise. DoS against anything you do not own is a criminal act in almost every jurisdiction, and it does not stop being one because it was 'just a test'.

2. The Cyber Kill Chain

Seven stages, in order. The value of the model is not memorising the names; it is being able to say what the defender does at each stage. Attackers must succeed at every stage; defenders only need to break one.

#	Stage	What the attacker does	What the defender does
1	Reconnaissance	Studies the target: public data, DNS, exposed services, employee names	Reduce public exposure, monitor certificate transparency and lookalike domains
2	Weaponisation	Builds the tool: a document, an exploit, a phishing page	Threat intelligence, blocking known signatures
3	Delivery	Sends it: email, USB, web link, compromised site	Email filtering, DMARC, user reporting, web filtering
4	Exploitation	Triggers the code or steals the credential	Patching, MFA, memory protection, least privilege
5	Installation	Establishes persistence: service, startup item, scheduled task	EDR, application control, monitoring persistence locations
6	Command & Control	Calls out to the attacker over HTTPS or DNS to receive orders	Egress filtering, DNS monitoring, beacon detection
7	Actions on Objectives	Steals data, encrypts it, or damages operations	Backups, data-loss monitoring, immutable logging

Worked example — one attack, seven stages

A finance employee receives an email with an invoice attachment (delivery). Opening it triggers a macro (exploitation) which writes a scheduled task (installation) and beacons to a new domain over HTTPS (C2) so the attacker can read the mail archive (actions on objectives).

Now answer one question per stage: which control, inside this bootcamp's lab, would have stopped it? If you cannot name one at four or more stages, you are relying on a single control and you do not have a defence.

Why MITRE ATT&CK was mentioned alongside it

The kill chain tells you the phase. ATT&CK gives you the specific technique so two analysts can describe the same behaviour the same way. Kill chain is a story; ATT&CK is a dictionary. You will use both — the kill chain when you brief management, ATT&CK when you write a detection.

3. Virtualisation and your lab

Term	Meaning
Host	Your real physical computer
Guest	The virtual machine running on the host
Hypervisor	The software layer that runs guests: VMware Workstation, VirtualBox, Hyper-V
Image	A disk file you can copy, snapshot and destroy: .iso (installer), .vmdk/.vdi (disk)
Snapshot	A saved state you can return to after breaking something

Why we work in virtual machines, in order of importance:

- **Recoverability:** An attacked VM can be reverted to a snapshot in seconds. A real machine cannot.
- **Containment:** Malware never touches the host operating system.
- **Multiple roles:** Two or three machines on one laptop, attacking and defending each other.
- **Reproducibility:** Copying a VM file is cloning a computer — essential for forensic practice later.

Network modes — the most important slide of the day

Mode	What it does	When to use it	Risk
Bridge	The VM becomes a real machine on your home network with its own LAN address	Downloading updates; reaching a lab target on your LAN	The VM is visible to every device on your network
NAT	Virtual switch gives internet access, host acts as router; other LAN devices cannot reach the VM	Normal day-to-day work in Kali	Cannot be reached from other machines
Host-only	Virtual switch with no internet at all	Malware analysis; deliberate isolation tests	No updates, no internet
Internal network	A private switch shared only by named VMs	Multi-machine labs and malware weekends	Invisible unless you name it in each VM

LOOKING AHEAD Today we used NAT and bridge while building the lab. For weekend 5 (malware) you will move to host-only or an internal network, and you will PROVE isolation rather than assume it. A lab that leaks turns a training exercise into a real incident.

Your lab layout today

```
Physical host: Windows
 |-- Hypervisor: VMware Workstation / VirtualBox / WSL
     |-- Kali Linux     (attacker machine)
     |-- Ubuntu Server  (victim / target machine)
```

Item	Where we got it
Kali virtual machine images	kali.org/get-kali → Kali virtual machines
Ubuntu Server image	osboxes.org → Ubuntu Server
Windows ISO (for later)	Microsoft evaluation downloads
Disk formats	.iso = installer image; .vmdk/.vdi = the virtual disk itself

4. Linux: distributions, shell, users

4.1 The families of distributions

Family	Package tool	Examples	Why it matters to you
Debian	apt / .deb	Debian, Ubuntu, Kali, Parrot	Your whole lab is Debian-family, so you use apt
Red Hat	dnf / yum / .rpm	RHEL, CentOS Stream, Fedora, Rocky	Enterprise servers you meet at work; the commands differ
Arch	pacman	Arch, BlackArch	Rolling release; BlackArch is a security-focused Arch

Learning point: apt is not 'Linux'. It is the Debian-family package manager. On a Red Hat server the same task is dnf install. Never assume the command transfers.

4.2 Reading the prompt

```
kali@ALIENWARE:~$     kali@ALIENWARE:~#
 |    |       | |       |
user  hostname  ~  |      root shell ($ is normal user, # is root)
                   home directory
```

Symbol	Meaning
kali	The user you logged in as
@ALIENWARE	The hostname of the machine
~	Your home directory, shorthand for /home/kali
$	You are a normal user
#	You are root — every command runs with full power and no undo

4.3 su, sudo, and doing escalation properly

Command	What it really does	Which password it asks for	When to use it
su	Switch user; with no argument it switches to root	The password of the target user (root)	When you know the root password and the root account has one
su root	Same, explicitly to root	The root password	Same as above
sudo -i	Start a login shell as root through sudo	YOUR password	The normal way on Kali and Ubuntu
sudo <cmd>	Run one command as root	YOUR password	The safest and most common form

CORRECTION Two corrections from the live session: (1) sudo asks for YOUR password, not root's — that is the whole point, so no one needs the root password. (2) The condition is not 'the user must be a superuser'; the user must be in the sudo group (or listed in /etc/sudoers). Kali's default user is set up this way, which is why it 'just works'.

```
# verify your own situation
id
sudo -l
groups
# leave the root shell when you are done
exit
```

4.4 Privilege escalation — the concept you will live in for 10 weekends

Privilege escalation means going from a low-privilege account to a higher one. It is the goal of most real attacks after initial access, and it is a whole module later in this course.

Type	Example	Where you meet it
Horizontal	Getting another normal user's account	Credential theft between colleagues
Vertical	Getting root or administrator	SUID binaries, sudo misconfiguration, kernel exploit
Escape	Leaving a container or restricted shell	Later cloud and container sessions

5. Files, users, groups and permissions

5.1 The top of the filesystem

Path	What it is	Investigation value
/	The single root of everything	Everything else hangs from here
/home	Home directories for normal users	/home/kali holds the user's files, history, keys
/root	Root's own home	Did the attacker escalate and leave artefacts here?
/etc	Configuration files	Who can log in, what services run
/var/log	Log files	Who authenticated, what failed, when
/tmp	World-writable scratch	Malware loves /tmp
/var/www/html	Default Apache document root	What the web server is actually serving

5.2 Reading permissions

```
-rwxr-xr--  1 kali kali 1234 Oct 24 15:20 public.txt
 ||||||||||
 |||||||||+-- others: r-- (read only)
 ||||||+------ group:  r-x (read + execute)
 |||||+---------- owner:  rwx (all)
 +------------ file type: - file, d directory, l link
```

Letter	Means	On a file	On a directory
r	read	See the contents	List the names inside
w	write	Change the contents	Create, rename or delete entries
x	execute	Run it as a program	Enter it with cd

Numbers are just the same three bits written in decimal: r=4, w=2, x=1, added per audience.

```
rwx = 4+2+1 = 7     r-x = 4+0+1 = 5     r-- = 4+0+0 = 4     --- = 0
```

Mode	Symbolic	Meaning in one line
777	rwxrwxrwx	Everyone can do everything. Almost always wrong.
755	rwxr-xr-x	Owner full, everyone else read+enter. The normal mode for directories and executables.
644	rw-r--r--	Owner read/write, everyone else read. The normal mode for files.
600	rw-------	Only the owner. The correct mode for private keys.
4755	rwsr-xr-x	755 plus SUID: runs with the owner's privileges. You will abuse these later.
000	---------	Nobody has any permission, including the owner.

5.3 chmod in both styles, exactly as in the lab

```
# what we ran in /var/www/html
sudo chmod o-r public.txt      # remove read from 'others'
ls -l public.txt               # verify the change

# symbolic: who + action + what  (u user, g group, o others, a all)
sudo chmod u+x script.sh
sudo chmod g-w shared.txt
sudo chmod o=r public.txt
sudo chmod a+r notice.txt

# octal: the same changes expressed as numbers
sudo chmod 640 report.txt      # rw- r-- ---
sudo chmod 755 script.sh       # rwx r-x r-x
sudo chmod 600 private_key.pem # rw- --- ---
```

```
OUTPUT
-rw-r--r-- 1 root root 0 Oct 24 14:41 public.txt      # before
-rw-r----- 1 root root 0 Oct 24 14:41 public.txt      # after chmod o-r
```

In the lab, chmod o-r made the file unreadable to other users — and then rm failed for that user, which showed something important: deleting a file is a permission on the DIRECTORY, not on the file.

WHY THE rm FAILED That last sentence is the single most useful thing in this section. A file with 000 permissions can still be deleted by anyone who can write to its directory. This is why world-writable directories are dangerous even when every file inside them is locked down.

Practice calculator: chmod-calculator.com. Use it to check yourself, never to learn the numbers.

5.4 Users and groups

```
sudo adduser zain              # interactive: creates user, group, home, asks for password
sudo useradd -m zain           # low-level: creates the account; YOU must set password and shell
id zain
cat /etc/passwd | grep zain
groups zain
```

OUTPUT

```
uid=1001(zain) gid=1001(zain) groups=1001(zain)
zain:x:1001:1001:,,,:/home/zain:/bin/bash
zain : zain
```

Field in /etc/passwd	Meaning
zain	username
x	password placeholder — the real hash lives in /etc/shadow
1001	UID
1001	GID
/home/zain	home directory
/bin/bash	login shell

adduser is the friendly Debian wrapper; useradd is the raw tool. If you create a user with useradd and forget -m and the shell, you get an account that cannot log in — that is a common real finding.

5.5 Package management

```
sudo apt update          # refresh the package list (what is available)
sudo apt upgrade -y      # install newer versions of what you already have
sudo apt install apache2
sudo apt remove apache2
sudo apt search wireshark
```

Command	What it touches	Common mistake
apt update	The package index only	Thinking it upgrades software — it does not
apt upgrade	Installed packages	Running update and skipping upgrade, then wondering why versions are old
apt install	Adds a package	Running install without update first, so the index is stale
apt remove	Removes the package, keeps config	Expecting config to disappear; use purge for that

6. Services and the web server lab

6.1 systemctl — controlling a service

```
sudo apt install apache2
sudo systemctl start apache2
sudo systemctl status apache2
sudo systemctl stop apache2
sudo systemctl enable apache2     # start automatically at boot
sudo systemctl restart apache2    # stop then start
sudo systemctl is-active apache2
```

Verb	Effect
start	Start now
stop	Stop now
restart	Stop then start — use after a config change
status	Show state plus the last lines of the service log
enable	Start automatically at boot

disable	Do not start at boot
is-active	One-word answer, useful in scripts

6.2 The document root and what the web server is allowed to read

Apache serves files out of /var/www/html by default. In the lab we erased the default page, wrote our own file, then changed its permissions and watched the server's behaviour change. That sequence is the whole lesson of this section.

```
cd /var/www/html
ls -l
sudo rm *                  # remove the default page (careful: rm * is permanent)
sudo nano /var/www/html/public.txt
sudo chmod o-r public.txt     # now 'others' cannot read it
sudo systemctl stop apache2
sudo systemctl start apache2
```

TRAP sudo rm * in a directory is exactly as destructive as it looks. Read the output of pwd and ls before you press enter. Every professional has a story about running a wildcard in the wrong directory; the difference is that they only did it once.

6.3 HTTP status codes — the five families

Range	Family	The ones you must know	What it tells an investigator
1xx	Informational	100 Continue, 101 Switching Protocols	Rare in investigations; protocol upgrades show here
2xx	Success	200 OK, 201 Created, 204 No Content	The request worked — a 200 on /admin is worth asking about
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified	Login redirects, and evidence of probing
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests	403/404 floods in access.log are the classic scanning signature
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable	A 500 appearing right after odd input often means the application choked on it

The three codes used in the session: 200 OK (served), 403 Forbidden (server refused — often a permission problem like the chmod we just did), 404 Not Found (not there). Reference for the full list: developer.mozilla.org → HTTP → Status → Server error responses.

WHY 403 MATTERS 403 after chmod o-r is not a bug, it is the web server correctly refusing to read a file its user cannot read. When you later break a web application on purpose, 403 is information: something exists and is protected. 404 is silence. Learning to read the difference is half of web reconnaissance.

7. Processes, logs and evidence

7.1 Processes

```
top                  # live view; q to quit
ps                   # your processes, current terminal
ps -A                # every process on the system
ps aux               # every process, with user and resource usage
ps -A | grep firefox     # filter for one program
kill 23269           # polite termination request (SIGTERM)
kill -9 23269        # force (SIGKILL) — no cleanup, no chance to save
```

Signal	Number	Meaning	Use when
SIGTERM	15	Ask the program to stop and tidy up	First attempt, always
SIGKILL	9	Kill immediately, cannot be caught	The process ignores SIGTERM
SIGHUP	1	Often used to reload configuration	After editing a service config

Windows equivalents: tasklist lists processes, taskkill /F /PID 35432 force-kills one. The /F is the Windows equivalent of -9.

7.2 The two Linux log files we read

```
cd /var/log
ls
cd apache2
cat access.log
tail -n 20 access.log        # last 20 lines, the useful view
cat error.log
```

```
OUTPUT
127.0.0.1 - - [24/Oct/2026:14:52:01 +0500] "GET /public.txt HTTP/1.1" 200 12 "-" "Mozilla/5.0"
127.0.0.1 - - [24/Oct/2026:14:52:03 +0500] "GET /missing.html HTTP/1.1" 404 197 "-" "Mozilla/5.0"
```

Field	Meaning	Investigation value
127.0.0.1	Client address	Who asked; repeated odd sources mean scanning
- -	Identity and username fields (usually empty)	Unreliable, do not rely on them
[date:time]	Timestamp with offset	The anchor for any timeline
"GET /public.txt HTTP/1.1"	Method, path, protocol	What they tried to reach
200	Status code	Success or failure of the attempt
12	Bytes returned	Large responses from odd paths are worth a look
"Mozilla/5.0"	User agent	Tools often lie, but a scanner's agent leaks it

Four commands that turn access.log into a finding

```
sudo awk '{print $9}' /var/log/apache2/access.log | sort | uniq -c | sort -rn   # status code counts
sudo grep -c ' 404 ' /var/log/apache2/access.log                                 # how many misses
sudo awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head   # busiest clients
sudo grep ' 500 ' /var/log/apache2/access.log | tail                             # server errors
```

A high 404 count from one address is usually a scanner. A 200 on a path nobody should be requesting is worth investigating properly. A burst of 500s right after unusual input is often the application failing on something an attacker sent.

WINDOWS SIDE On the Linux side the authentication log is /var/log/auth.log or journalctl; on Windows it is the Security event log with Event ID 4624 for successful logon and 4625 for failed logon. Repeated 4625 then a 4624 for the same account is brute force that succeeded — the single most important pattern in entry-level SOC work, and you will meet it again in weekend 6.

7.3 The command history is evidence

In the session we ran history to see the day's commands. That file is one of the most useful artefacts in an investigation, because it records what a user actually did, in order.

```
history          # this session's commands
history 30       # last 30
cat ~/.zsh_history | tail -30     # Kali's default shell is zsh
cat ~/.bash_history | tail -30    # bash systems
```

Artefact	Where	What it proves
Shell history	~/.bash_history or ~/.zsh_history	Commands typed by that user (not by scripts)
Auth log	/var/log/auth.log or journalctl	Who logged in, from where, which method
Apache access log	/var/log/apache2/access.log	What was requested, by whom, with what result
Process table	ps / top / /proc	What is running right now
Timestamps on files	ls -l / stat	When something was created or modified

8. Networking essentials

8.1 IP addresses and classes

Class	First octet	Private range	Structure	Hosts per network
A	1–126	10.0.0.0 – 10.255.255.255	N.H.H.H	16,777,214
B	128–191	172.16.0.0 – 172.31.255.255	N.N.H.H	65,534
C	192–223	192.168.0.0 – 192.168.255.255	N.N.N.H	254
IPv6	—	fc00::/7	128 bits, written in hex groups	Effectively unlimited

ADDRESSES YOU WILL SEE TODAY 127.x.x.x is loopback (this machine). 169.254.x.x is link-local — if a computer has that address, DHCP failed, and it is a classic first thing to check. 0.0.0.0 in a listening socket means every interface.

8.2 Ports

Range	Name	Notes
0–1023	Well-known	Need root to bind on Linux
1024–49151	Registered	Databases and applications live here
49152–65535	Dynamic / ephemeral	The source port of your outbound connection

Port	Service	Port	Service
21	FTP	53	DNS (TCP and UDP)
22	SSH	67/68	DHCP server / client
23	Telnet (plaintext — never use it)	69	TFTP
25	SMTP	80	HTTP
123	NTP	1433	MSSQL
161/162	SNMP	443	HTTPS
514	Syslog	3306	MySQL

8.3 ping, TTL, and estimating distance

```
ping -c 4 google.com
ip a
ip addr
ifconfig          # older tool, still common on other systems
traceroute google.com
```

OUTPUT

```
PING google.com (142.250.x.x) 56(84) bytes of data.
64 bytes from 142.250.x.x: icmp_seq=1 ttl=115 time=24.3 ms
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
```

How TTL actually works

Fact	Detail
TTL is a hop counter	Each router decrements it by one. It starts at an OS default and you see what is left
Typical starting values	Linux and most network devices: 64. Windows: 128. Some devices: 255
Reading ttl=115	≤ 128 and > 64, so the start was 128 → 128 − 115 = 13 hops
Reading ttl=52	≤ 64, so the start was 64 → 64 − 52 = 12 hops
Reliability	It is an estimate. Devices can rewrite TTL, NAT can change it, tunnels hide hops

CORRECTION — READ THIS TTL is triage, not proof. In an investigation it is a hint that costs nothing to check; it is not evidence of the operating system. A firewall appliance rewriting TTL will make a Windows host look like Linux and you will write a wrong finding if you forget that.

9. Ciphers, encoding and encryption

9.1 The distinction that matters

Term	What it does	Reversible without a key?	Example
Encoding	Preserves data in a different format for transport	Yes — always	Base64, URL encoding
Cipher (classical)	Hides meaning with a scheme	Yes for modern attackers (Caesar takes seconds)	Caesar, Vigenère
Hashing	Produces a fixed-length fingerprint	No — one-way by design	SHA-256, MD5
Encryption	Hides data behind a mathematical key	Only with the correct key	AES, RSA
Steganography	Hides the data's existence	With the right tool	Image LSB, audio

9.2 Caesar cipher — HACKER became KDFNHU

A Caesar cipher shifts every letter by a fixed amount. Today you demonstrated +3:

Plain	H	A	C	K	E	R
Shift +3	K	D	F	N	H	U

```
# verify the shift yourself
python3 -c "s='HACKER'; print(''.join(chr((ord(c)-65+3)%26+65) for c in s))"
python3 -c "s='KDFNHU'; print(''.join(chr((ord(c)-65-3)%26+65) for c in s))"
```

```
OUTPUT
KDFNHU
HACKER
```

Why it is useless as security: only 25 possible keys. A computer tries all of them instantly, and a human can brute-force it by eye. Its value today is that it teaches the concept of a key and a substitution.

Morse code — also encoding, and also in the session

h	e	l	l	o
....	.	.-..	.-..	---

9.3 Base64 — encoding, not encryption

```
echo -n 'hello world' | base64
echo -n 'hello world' | base64 | base64 -d
echo 'hello world' | base64       # note the difference
```

```
OUTPUT
aGVsbG8gd29ybGQ=
aGVsbG8gd29ybGQK     # because echo adds a newline
hello world
```

THE TRAILING NEWLINE In the session we saw aGVsbG8gd29ybGQK for 'hello world'. The trailing K is a newline character that echo added. Use echo -n (or printf) when you want to encode exactly what you typed. This off-by-one-newline problem causes real authentication failures in scripts and real false findings in investigations.

Base64 exists because some channels only carry text. It is reversible by anyone, with no key. Finding Base64 in a payload tells you how it was transported, not that it was secret. When you meet it in a malware sample later, the question is 'decode it and read the next stage', not 'can it be cracked'.

```
# the practical test: can you recover it without a key?
echo 'aGVsbG8gd29ybGQ=' | base64 -d     # yes, instantly
```

9.4 Symmetric encryption with OpenSSL, exactly as we ran it

```
mkdir cyber && cd cyber
nano secret.txt                    # type something, e.g. 'my secret message'

openssl enc -aes-256-cbc -salt -in secret.txt -out secret.enc
ls -l secret.txt secret.enc
cat secret.enc                     # unreadable garbage

openssl enc -d -aes-256-cbc -in secret.enc -out decrypt.txt
cat decrypt.txt                    # your original text
```

OUTPUT

```
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
(secret.txt and secret.enc, the second one binary)
my secret message
```

Flag	Meaning
enc	The symmetric cipher tool inside OpenSSL
-aes-256-cbc	The cipher: AES with a 256-bit key in Cipher Block Chaining mode
-salt	Adds a random salt so the same password produces different ciphertext each time
-in / -out	Input and output files
-d	Decrypt instead of encrypt

UPGRADE YOUR COMMAND Two improvements on what we ran today. (1) Add -pbkdf2: modern OpenSSL warns that the default key derivation is deprecated (we verified this on OpenSSL 3.6.3). (2) CBC does not detect tampering — flip a byte and you get silent garbage. For real work use an authenticated mode (GCM) through a library, because the openssl enc command does not support AEAD ciphers at all: it fails with 'AEAD ciphers not supported'.

9.5 Looking inside the file

```
xxd -l 16 secret.enc
file secret.enc
```

OUTPUT

```
00000000: 5361 6c74 6564 5f5f a7a0 ff99 589c 686d  Salted__....X.hm
secret.enc: openssl enc'd data with salted password
```

Salted__ is an 8-byte magic header that OpenSSL writes when -salt is used. It is not secret — it simply tells the decryption step which salt to use. In forensics, that header plus the file command is how you identify an unknown blob in seconds.

9.6 Asymmetric encryption: RSA, exactly as we ran it

```
openssl genrsa -out Private_Key.pem 4096
cat Private_Key.pem | head -3
openssl rsa -in Private_Key.pem -pubout -out Public_Key.pem
cat Public_Key.pem

ls -l Private_Key.pem Public_Key.pem

openssl pkeyutl -encrypt -in secret.txt -out encrypted.rsa -pubin -inkey Public_Key.pem
openssl pkeyutl -decrypt -in encrypted.rsa -out decrypted.txt -inkey Private_Key.pem
cat decrypted.txt
```

OUTPUT

```
-----BEGIN PRIVATE KEY-----
-----BEGIN PUBLIC KEY-----
-rw------- 1 kali kali 3272 ... Private_Key.pem
-rw-r--r-- 1 kali kali  800 ... Public_Key.pem
my secret message
```

Concept	Meaning in this lab
Public key	Used to ENCRYPT. Safe to share. This is why pkeyutl uses -pubin -inkey Public_Key.pem
Private key	Used to DECRYPT. Never share it. It is created with mode 600 for a reason

4096	Key size in bits. Signature and ciphertext length follow the modulus (4096 bits = 512 bytes)
pkeyutl	The generic public-key operation tool; encrypt and decrypt are separate -in/-out pairs

THE LIMIT YOU MUST KNOW RSA can only encrypt a small amount of data — roughly the key size minus padding. We verified that a 406-byte file fails with 'data too large for key size'. This is why real systems use hybrid encryption: RSA protects a random AES key, and AES encrypts the data. You will see this pattern again in TLS.

Symmetric versus asymmetric, in one table

	Symmetric (AES)	Asymmetric (RSA)
Keys	One shared key	A key pair: public + private
Speed	Fast, used for bulk data	Slow, used for small data or key exchange
Problem it solves	Bulk encryption	The key distribution problem
In TLS	Encrypts the actual traffic	Authenticates the server and exchanges the session key

10. The security vocabulary — fill this in

These five words appear in every report you will write for the rest of the course. The definitions below are the ones to use.

Term	Definition	Our running example
Asset	Something of value you are protecting	The Apache web server and the files in /var/www/html
Threat	Who or what could act against the asset	A scanner looking for exposed services
Vulnerability	The weakness a threat can use	A file in the web root that should not be public
Risk	Likelihood × impact, as a decision	Medium: exposure is real, impact is limited
Exploit	The working method that produces an effect	A single GET request for that file

Term	Definition (write your own)
Asset	
Threat	
Vulnerability	
Risk	
Exploit	

Proof of concept (PoC)

A PoC is the smallest working demonstration that a vulnerability is real: the request, the script, the screenshot, the output. It is what separates a finding from an opinion. In every report you write in this course, a finding without a PoC or a log line is a guess.

TRAP Do not put 'risk: CVE-2024-xxxx' in a report. A CVE identifier is not a risk. Risk needs the asset, the threat, the exposure and the impact, and it needs a recommendation with a cost.

11. Shell essentials

11.1 Chaining operators

Operator	Meaning	Example	Result
;	Run the next command regardless of the first	ping -c1 x ; echo done	done always prints
|	Send stdout of the first into the second	ps -A | grep firefox	Only matching lines
&&	Run the next command only if the first succeeded	apt update && apt upgrade	Upgrade only if update worked
&	Run in the background	wireshark&	Prompt returns immediately

```
true ; echo after-semicolon
false && echo not-printed || echo or-printed
wireshark&          # then: jobs, fg, kill %1
```

11.2 Help is always available

```
ls --help | head -20
man ls                  # press q to quit
apropos chmod               # search the manuals
openssl enc -help 2>&1 | head
ls -h                   # NOTE: here -h means human-readable, not help!
```

TRAP On the live session you tried 'command -h' and 'command --h'. Careful: -h is not help for every command. For ls it means human-readable sizes. The reliable patterns are --help and man.

11.3 Aliases

```
alias ll='ls -l'
alias l='ls -lah'
alias ..='cd ..'
alias                  # list current aliases
unalias ll               # remove one
# permanent: add the alias lines to ~/.zshrc (Kali) or ~/.bashrc
```

In the session, ll appeared to work. On Kali zsh this is often already an alias or a function, which is why it seemed built in. Check with 'alias' and 'type ll' before you assume.

11.4 Shells: sh, bash, zsh

Shell	Path	What it is	Where you meet it
sh	/bin/sh	The Bourne shell, the POSIX baseline	Scripts that must run anywhere
bash	/bin/bash	Bourne Again SHell — the most common login shell	Most Linux servers
zsh	/bin/zsh	Z shell — a bash-compatible shell with more features	Kali's default
fish	/usr/bin/fish	Friendly interactive shell, not POSIX	Personal desktops

```
echo $SHELL
echo $0
cat /etc/passwd | grep kali | cut -d: -f7
```

Why it matters later: a script written for bash may fail under sh (dash), and Kali's zsh history file is .zsh_history, not .bash_history. When you go looking for evidence in weekend 8, knowing which shell wrote the history is the difference between finding it and reporting that it does not exist.

11.5 Windows password storage — mentioned today, explained

Windows stores local account password hashes in the SAM database at C:\Windows\System32\config\SAM, in NTLM format. NTLM is a hash, not encryption: it cannot be reversed, so tools such as John the Ripper and Hashcat guess candidate passwords and compare hashes.

Hash type	Produced	Why it is weak
LM	Legacy Windows	Split into two 7-character halves, convert to uppercase: broken instantly
NTLM	Modern Windows	No salt, no stretching — a fast GPU tests billions of guesses per second
NTLMv2 (network)	Authentication over the network	Captured from the wire and cracked offline; also relayable

SCOPE — NON-NEGOTIABLE Never point John or Hashcat at a hash you did not obtain legally. In this course you will crack hashes from your own lab machine and from deliberately provided practice sets only. Credential theft and cracking of real user data is a criminal offence, and it is also a guaranteed way to end a career that has not started yet.

12. Research work — homework

Two tasks. Both are marked on evidence and specificity, not on length. Hand in as a markdown file in your GitHub repository under /s01-foundations.

Task A (compulsory) — Malware types and examples

Produce a table with one row per malware type. For each, give a real named example with the year, who it targeted, how it spread, and what it did. Then write three sentences on which type you think is most dangerous for a Pakistani SME and why.

Type	Definition in your own words	Named example + year	How it spread	Primary impact
Virus				
Worm				
Trojan				
Ransomware				
Spyware				
Adware				
Rootkit				
Keylogger				
Botnet agent				
Fileless / living-off-the-land				
Logic bomb				
Bootkit				

HOW THIS IS MARKED Do not copy definitions from a vendor blog. If the sentence would make sense with the malware name deleted, it is too generic. The marked quality is whether a reader who knows the field would learn something from your row.

Task B (choose any two) — pick two, write up properly

Option	Task	Evidence to include
B1	Take a screenshot of your own VM proving the lab is isolated: interfaces, route, and a failed ping to 8.8.8.8	Three terminal screenshots with the address bar visible, plus one sentence on what each proves
B2	Create a file, change its permissions three ways with chmod (symbolic and octal), and explain what each change did	ls -l before and after each change, with the octal value labelled
B3	Write a Caesar cipher decoder in Python or bash that takes a shifted string and prints all 25 possible outputs	The script, one sample run, and one line explaining why 25 tries is not security
B4	Encrypt a file with OpenSSL, decode its header with xxd, and explain what Salted__ means	The commands, the xxd output, and your explanation of the header
B5	Write a one-page MITM case study of a real incident: who, how, what the attacker got, what control would have stopped it	Cited source, the control, and the cost of that control
B6	Look up your own public IP, identify which address class it belongs to, and trace the route to a website with traceroute	The outputs, and your hop-count estimate from TTL, including one reason TTL can mislead

Submission standard

- **Format:** Markdown file, your own words, screenshots embedded.
- **Evidence:** Every claim has a command, a screenshot, or a cited source next to it.
- **Delivery:** Commit and push to GitHub; put the link in the cohort channel.
- **Deadline:** Tuesday 27 October 2026, 23:59.

13. Quick reference — everything today on two pages

Task	Command
Time zone	timedatectl | timedatectl list-timezones | sudo timedatectl set-timezone Asia/Karachi
Become root	sudo -i (your password) | sudo -s | su - (root password)
Where am I / who am I	pwd | whoami | id
List files	ls | ls -l | ls -la | ls -lah
Home	cd | cd ~ | cd - (previous directory)
Create/remove	mkdir -p a/b | touch f | rm f | rm -r dir | rmdir dir
Copy/move	cp -r src dst | mv old new
Permissions	chmod 755 f | chmod o-r f | chown user:group f | stat -c '%a %A' f
Users	sudo adduser name | id name | passwd name | groups name
Packages	apt update | apt upgrade | apt install pkg | apt remove pkg | apt search term
Services	systemctl start|stop|restart|status|enable|disable name
Processes	top | ps -A | ps aux | ps -A | grep name | kill PID | kill -9 PID
Windows process	tasklist | taskkill /F /PID 1234
Logs	cat /var/log/apache2/access.log | tail -n 20 | journalctl | history
Network info	ip a | ping -c 4 host | traceroute host | ss -tulpn
Encode	echo -n text | base64 | echo CODE | base64 -d
Hash	sha256sum file | md5sum file
Symmetric encrypt	openssl enc -aes-256-cbc -salt -pbkdf2 -in f -out f.enc (decrypt: add -d)
Inspect a blob	xxd -l 16 f.enc | file f.enc
Asymmetric	openssl genrsa -out priv.pem 4096 | openssl rsa -in priv.pem -pubout -out pub.pem
RSA encrypt/decrypt	openssl pkeyutl -encrypt -in f -out e -pubin -inkey pub.pem | pkeyutl -decrypt -inkey priv.pem
Help	man cmd | cmd --help | apropos term
Chaining	cmd1 ; cmd2 (always) | cmd1 && cmd2 (on success) | cmd1 | cmd2 (pipe) | cmd & (background)

Ports cheat sheet

Port	Service	Port	Service	Port	Service
21	FTP	53	DNS	443	HTTPS
22	SSH	67/68	DHCP	514	Syslog
23	Telnet	69	TFTP	1433	MSSQL
25	SMTP	80	HTTP	3306	MySQL
123	NTP	161/162	SNMP	3389	RDP

The five things to remember from Day 1

1. Controls protect confidentiality, integrity or availability — name which one before you recommend it.
2. The kill chain has seven stages; defence means breaking one of them, not all of them.
3. Permissions are three numbers, and deleting a file depends on the directory, not the file.
4. Encoding is not encryption, and a hash cannot be decrypted.
5. Every claim needs a command, a screenshot or a log line behind it — otherwise it is a guess.

Next session: Operating Systems in depth, Windows event logs, and your first log investigation. Bring the same VM, snapshotted.